

Linear Search & Binary Search Quiz

A comprehensive 50-question MCQ quiz focusing exclusively on Linear Search and Binary Search in Data Structures & Algorithms. Total Points: 50.

aasthalakhataria7@gmail.com [Switch accounts](#)



Not shared

* Indicates required question

1. What is the fundamental approach of the Linear Search algorithm? *

- ☐ Divides the array into halves recursively
- ☐ Checks each element sequentially from start to end until a match is found
- ☐ Jumps ahead by fixed steps to find the target
- ☐ Hashes the key to find the exact bucket

2. In the worst-case scenario, how many comparisons does Linear Search perform on an array of size N? *

- ☐ 1
- ☐ $\log_2(N)$
- ☐ $N / 2$
- ☐ N



3. What is the best-case time complexity of Linear Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$

4. Under what condition is Linear Search the most optimal or only viable choice among searching algorithms? *

- ☐ When the data is strictly sorted in descending order
- ☐ When the data structure is unsorted and cannot be modified or reindexed
- ☐ When the data size exceeds millions of elements
- ☐ When memory availability is completely unrestricted

5. What is the average-case time complexity of Linear Search on an unsorted array of size N ? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N^2)$



6. What is the auxiliary space complexity of a standard iterative Linear Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N^2)$

7. If an item is present at the exact middle of an unsorted array, how many comparisons will Linear Search take? *

- ☐ 1 comparison
- ☐ $\log_2(N)$ comparisons
- ☐ $N / 2$ comparisons (or nearby integer)
- ☐ N comparisons

8. Which data structure allows Linear Search to perform with a space complexity of $O(1)$? *

- ☐ Singly Linked List
- ☐ Arrays
- ☐ Doubly Linked List
- ☐ All of the above



9. What does a Linear Search function typically return if the element is not present in the collection? *

- ☐ 0
- ☐ 1
- ☐ -1 (or an invalid index/null object)
- ☐ Throws a segmentation fault

10. Can Linear Search be performed on a sorted array? *

- ☐ No, it will cause an infinite loop
- ☐ Yes, but it is less efficient than Binary Search
- ☐ Yes, and it performs faster than Binary Search on large arrays
- ☐ No, Linear Search syntax prevents it

11. What is the core paradigm or strategy behind Binary Search? *

- ☐ Greedy Approach
- ☐ Dynamic Programming
- ☐ Divide and Conquer
- ☐ Brute Force



12. What is the absolute prerequisite for executing a standard Binary Search on an array? *

- ☐ The array elements must be unique
- ☐ The array must be completely sorted
- ☐ The array must contain only positive integers
- ☐ The array size must be a power of 2

13. What is the worst-case time complexity of Binary Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$

14. What is the best-case time complexity of Binary Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(1.5)$



15. In Binary Search, if the target element is smaller than the middle element, where do you look next? *

- ☐ In the right half of the array
- ☐ In the left half of the array
- ☐ You restart from the beginning using Linear Search
- ☐ The search terminates immediately

16. What is the auxiliary space complexity of an iterative implementation of Binary Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$

17. What is the space complexity of a recursive implementation of Binary Search due to call stack overhead? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N^2)$



18. For an array of size 1024, what is the maximum number of comparisons Binary Search will make? *

- ☐ 10
- ☐ 11
- ☐ 512
- ☐ 1024

19. Which formula is safest to calculate the mid-index in Binary Search to prevent integer overflow in languages like C++ or Java? *

- ☐ $\text{mid} = (\text{low} + \text{high}) / 2$
- ☐ $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$
- ☐ $\text{mid} = (\text{low} * \text{high}) / 2$
- ☐ $\text{mid} = \text{high} - (\text{low} + \text{high}) / 2$

20. If Binary Search is applied to a sorted list of 1 million elements, roughly how many comparisons are needed in the worst-case scenario? *

- ☐ Around 20
- ☐ Around 500
- ☐ Around 50,000
- ☐ Around 500,000



21. Why is Binary Search faster than Linear Search for large datasets? *

- ☐ It uses multiple processor cores simultaneously
- ☐ It eliminates half of the remaining elements from consideration at each step
- ☐ It reads memory locations faster
- ☐ It converts numbers into strings before processing

22. When an array has only 3 sorted elements, which algorithm is generally faster *
in execution due to lower initialization overhead?

- ☐ Binary Search
- ☐ Linear Search
- ☐ Both take exactly the same microsecond time
- ☐ Neither can run on 3 elements

23. Suppose you have a dynamic list where insertions and deletions happen constantly, and you only perform occasional searches. Which combination makes the most sense? *

- ☐ Keep list unsorted; use Linear Search
- ☐ Sort the list after every single insertion; use Binary Search
- ☐ Keep list sorted; use Linear Search
- ☐ Convert list to an array stack; use iterative Binary Search



24. Binary Search cannot be implemented efficiently (retaining $O(\log N)$ time) on which of the following data structures even if it is sorted? *

- ☐ Static Array
- ☐ Dynamic Array (ArrayList)
- ☐ Singly Linked List
- ☐ Vector

25. What is the time complexity of searching an element in a sorted Singly Linked List using Binary Search? *

- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$
- ☐ $O(N^2)$

26. Linear Search is classified as a _____ algorithm, while Binary Search is a _____ algorithm. *

- ☐ Sequential, Logarithmic
- ☐ Logarithmic, Exponential
- ☐ Randomized, Deterministic
- ☐ Sort-driven, Hashed



27. If an array of 10 elements is sorted, what is the maximum number of comparisons for Linear Search vs. Binary Search? *

- ☐ LS: 10, BS: 10
- ☐ LS: 10, BS: 4
- ☐ LS: 5, BS: 2
- ☐ LS: 1, BS: 4

28. Which algorithm is inherently stable if we want to find the first occurrence of a duplicate item in an unsorted array? *

- ☐ Binary Search
- ☐ Linear Search from left to right
- ☐ Binary Search modified with pointers
- ☐ None of the above

29. If you sort an unsorted array first using Merge Sort $O(N \log N)$ and then perform a single Binary Search $O(\log N)$, what is the total time complexity? *

- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$
- ☐ $O(N^2)$



30. Under what frequency of searching is the cost of sorting an array for Binary Search justified? *

- ☐ When you only need to search once
- ☐ When you need to search multiple times (Highly repetitive lookups)
- ☐ When you never search at all
- ☐ Sorting is never justified

31. In a modified Binary Search targeting the 'First Occurrence' of an element in a duplicate-filled sorted array, what do you do when `arr[mid] == target`? *

- ☐ Return mid immediately
- ☐ Set `high = mid - 1` and keep searching in the left half
- ☐ Set `low = mid + 1` and keep searching in the right half
- ☐ Terminate the program with an error

32. In a modified Binary Search targeting the 'Last Occurrence' of an element in a duplicate-filled sorted array, what do you do when `arr[mid] == target`? *

- ☐ Return mid immediately
- ☐ Set `high = mid - 1` and keep searching in the left half
- ☐ Set `low = mid + 1` and keep searching in the right half
- ☐ Change target value dynamically



33. What happens if you run a standard Binary Search loop with the condition `while (low < high)` instead of `while (low <= high)`? *

- ☐ It will cause a compiler error
- ☐ It might miss the target if it resides at the final remaining single-element boundary
- ☐ It will run into an infinite loop every time
- ☐ It behaves exactly the same with no exceptions

34. What is the behavior of Linear Search when looking for an element in an array containing all identical elements? *

- ☐ It throws an array out of bounds exception
- ☐ It finds the element at index 0 in 1 comparison
- ☐ It goes to the end of the array every time
- ☐ It runs in $O(\log N)$ time

35. Which optimization can be applied to Linear Search to speed up subsequent searches of the same element? *

- ☐ Move the found element to the front of the array (Move-to-Front heuristic)
- ☐ Sort the array backwards
- ☐ Delete the element after finding it
- ☐ Double the array size



36. What is the value of low and high after a standard Binary Search terminates unsuccessfully because the element is absent? *

- ☐ low > high
- ☐ low == high
- ☐ low == 0 and high == 0
- ☐ low and high both become negative

37. If you are doing a Binary Search on a sorted array in descending order, how do the pointer updates change? *

- ☐ They don't change at all
- ☐ If target < arr[mid], move low = mid + 1; else high = mid - 1
- ☐ If target < arr[mid], move high = mid - 1; else low = mid + 1
- ☐ You cannot perform Binary Search on descending arrays

38. How many comparisons does Linear Search take to confirm that an element is missing in an unsorted array of size N? *

- ☐ $\log_2(N)$
- ☐ $N / 2$
- ☐ N
- ☐ $N + 1$



39. In an ordered array, if you know the target is near the very beginning, which algorithm can you optimize Linear Search into? *

- ☐ Sentinel Linear Search
- ☐ Two-way Binary Search
- ☐ Exponential Search setup phase
- ☐ None of the above

40. What is a 'Sentinel Search' in the context of linear searching? *

- ☐ A search that uses graphics cards
- ☐ Placing the key at the end of the array to eliminate the loop boundary check condition ($i < N$) at each step
- ☐ Using two pointers from both ends of the array
- ☐ Using a random number generator to find indices

41. Can Binary Search be used to find the square root of a number? *

- ☐ No, Binary Search is strictly for arrays
- ☐ Yes, by defining a search space between 0 and the number and matching $mid * mid$ against target
- ☐ Yes, but only for numbers less than 10
- ☐ No, because square roots are irrational numbers



42. How does an infinite array search adapt Binary Search (often called Exponential or Infinite Search)? *

- ☐ It searches sequentially forever
- ☐ It increments indices exponentially (1, 2, 4, 8...) to find an upper bound, then applies standard Binary Search within that range
- ☐ It shrinks the array continuously
- ☐ It is mathematically impossible

43. If an array is rotated but sorted (e.g., [4, 5, 6, 7, 0, 1, 2]), can we still use a modified Binary Search? *

- ☐ No, rotation completely ruins the sorted property required for Binary Search
- ☐ Yes, by checking which half of the array is normally sorted at each step and narrowing down pointers
- ☐ Yes, but it reverts to $O(N)$ complexity structurally
- ☐ No, it causes memory corruption

44. What is the time complexity of finding the peak element in a mountain array (strictly increasing then strictly decreasing) using Binary Search? *

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$



45. If you pass an array to a recursive Binary Search function by value instead of by reference/pointer in languages like C++, what happens to the overall time complexity? *

- ☐ Stays $O(\log N)$
- ☐ Becomes $O(N)$ or $O(N \log N)$ because copying the array takes linear time at each recursive call stack frame
- ☐ Becomes $O(1)$
- ☐ Causes an immediate compilation syntax error

46. What is the primary drawback of using an iterative Binary Search over Linear Search on tiny datasets (e.g., 2 or 3 elements)? *

- ☐ It uses excessive memory
- ☐ The constant factors and initialization overhead (low, high, mid calculations) might make it marginally slower than a raw single-line loop check
- ☐ It fails to find elements on tiny screens
- ☐ There is no drawback; it is always vastly faster

47. In a successful Binary Search, what is the minimum number of comparisons needed to find an element? *

- ☐ 0
- ☐ 1 (if the target is exactly at the initial mid position)
- ☐ $\log_2(N)$
- ☐ $N / 2$



48. What occurs during a Linear Search if the search array is entirely empty? *

- ☐ It throws a NullPointerException immediately
- ☐ The loop condition fails initially, returning the default 'not found' index safely
- ☐ It loops infinitely
- ☐ It crashes the local operating system

49. If we perform Binary Search on a bitonic array (an array that first increases and then decreases), what must we do first? *

- ☐ We can run standard Binary Search directly
- ☐ Find the peak element using binary search, then binary search separately on both halves
- ☐ We must run linear search; binary variations fail completely
- ☐ Sort the entire array again using Bubble sort

50. If you have an array containing a single element, how many comparisons will Linear Search and Binary Search take respectively to find that element if it matches your target? *

- ☐ Linear: 1, Binary: 1
- ☐ Linear: 1, Binary: 0
- ☐ Linear: 0, Binary: 1
- ☐ Linear: 2, Binary: 1

Submit

Clear form

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#).

Does this form look suspicious? [Report](#)



Google Forms

